

Some Problems that can be solved using graphs

1. Suppose you run a day care for an office building and there are seven children A, B, C, D, E, F. we need to assign a locker where each child's parent can put the child's food. The children come and leave so they are not all there at the same time. You have 1 hour time slots starting 7:00 a.m. to 9:00 am. A star in the table means a child is present at that time. What is the minimum number of lockers necessary? Show how you would assign the lockers

	A	B	C	D	E	F
7:00	*	-	*	-	*	-
8:00	-	*	*	-	-	-
9:00	-	-	-	*	*	*

2. We want to schedule a training programs with following training modules for employees: A,B,C,D,E,F,G,H

Following pairs of training modules have no common employee:

A-B,B-C,A-D,A-E,B-E,E-F,A-F,B-F,D-F, A-G,A-H,B-H,C-H

How many minimum training slots are needed?

3. Map coloring:

Color the states so that no two adjacent states have same color. What is the minimum number of colors to be used?



Graph Algorithms: Breadth First Search

BFS(V, E, s)

for each $u \in V - \{s\}$

do $d[u] \leftarrow \infty$

$d[s] \leftarrow 0$

$Q \leftarrow \emptyset$

ENQUEUE(Q, s)

while $Q \neq \emptyset$

do $u \leftarrow \text{DEQUEUE}(Q)$

for each $v \in \text{Adj}[u]$

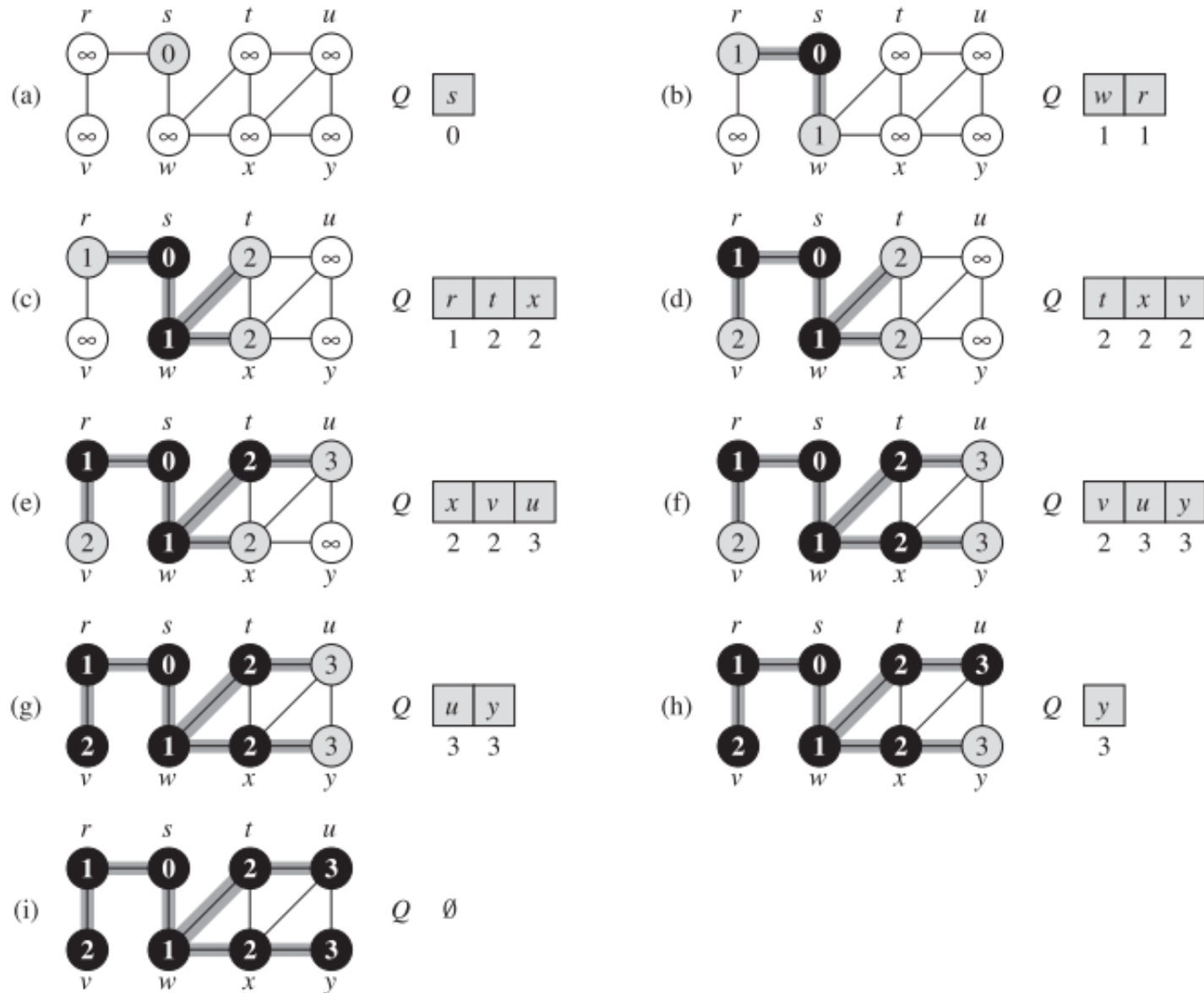
do if $d[v] = \infty$

then $d[v] \leftarrow d[u] + 1$

 ENQUEUE(Q, v)

Complexity = $O(|V| + |E|)$

BFS in action



Graph Algorithms: Depth First Search

DFS(V, E)

for each $u \in V$

do $color[u] \leftarrow \text{WHITE}$

$time \leftarrow 0$

for each $u \in V$

do if $color[u] = \text{WHITE}$

then DFS-VISIT(u)

Complexity
= $(|V| + |E|)$

DFS-VISIT(u)

$color[u] \leftarrow \text{GRAY}$ $\triangleright$ discover u

$time \leftarrow time + 1$

$d[u] \leftarrow time$

for each $v \in Adj[u]$ $\triangleright$ explore (u, v)

do if $color[v] = \text{WHITE}$

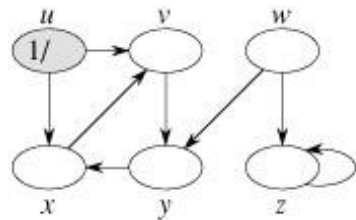
then DFS-VISIT(v)

$color[u] \leftarrow \text{BLACK}$

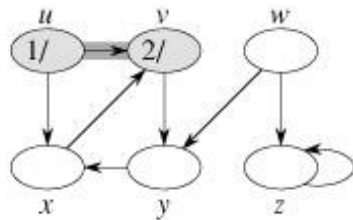
$time \leftarrow time + 1$

$f[u] \leftarrow time$ $\triangleright$ finish u

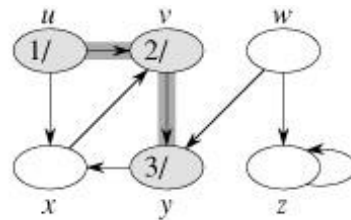
DFS in action



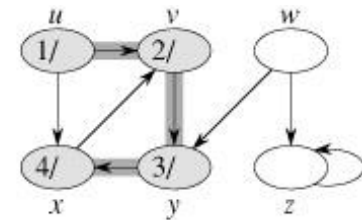
(a)



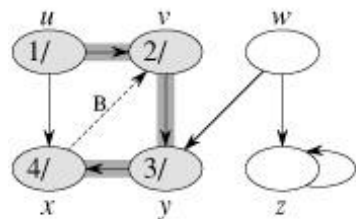
(b)



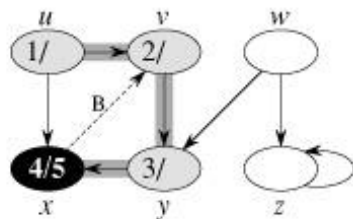
(c)



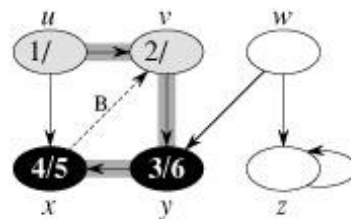
(d)



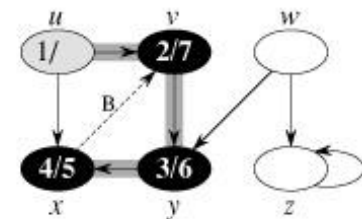
(e)



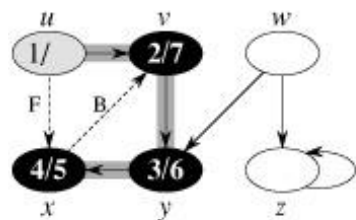
(f)



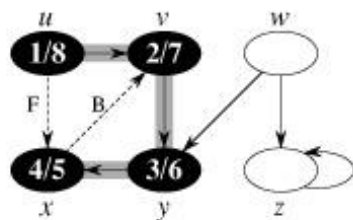
(g)



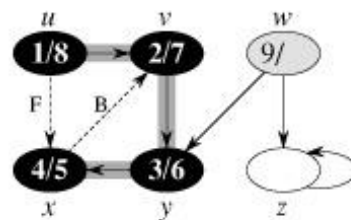
(h)



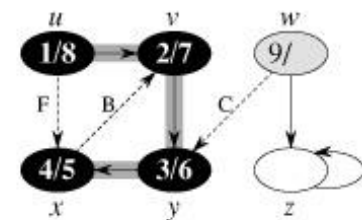
(i)



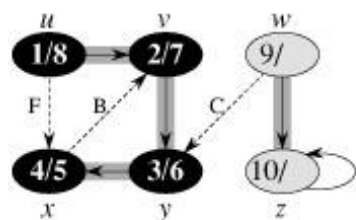
(j)



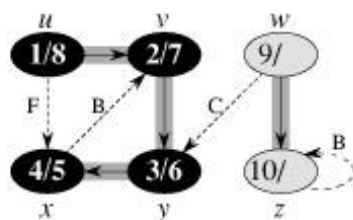
(k)



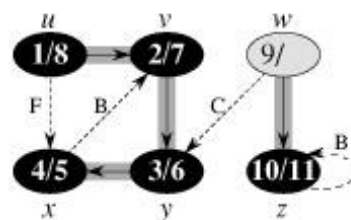
(l)



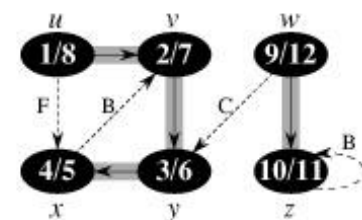
(m)



(n)



(o)



(p)

Graph Algorithms: Kruskal's MST

$G = (V, E)$ is a connected, undirected, weighted graph. $w : E \rightarrow \mathbf{R}$.

- Starts with each vertex being its own component.
- Repeatedly merges two components into one by choosing the light edge that connects them (i.e., the light edge crossing the cut between them).
- Scans the set of edges in monotonically increasing order by weight.
- Uses a disjoint-set data structure to determine whether an edge connects vertices in different components.

Graph Algorithms: Kruskal's MST (pseudoC)

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Time complexity = $O(m \log n)$

$|V| = n, |E| = m$

MAKE-SET(x)

```
1   $x.p = x$ 
2   $x.rank = 0$ 
```

UNION(x, y)

```
1  LINK(FIND-SET( $x$ ), FIND-SET( $y$ ))
```

LINK(x, y)

```
1  if  $x.rank > y.rank$ 
2       $y.p = x$ 
3  else  $x.p = y$ 
4      if  $x.rank == y.rank$ 
5           $y.rank = y.rank + 1$ 
```

FIND-SET(x)

```
1  if  $x \neq x.p$ 
2       $x.p = \text{FIND-SET}(x.p)$ 
3  return  $x.p$ 
```

See Chapter 21 in the book for Union and Find-Set operations

Graph Algorithms: Prim's MST

- Builds one tree, so A is always a tree.
- Starts from an arbitrary “root” r .
- At each step, find a light edge crossing cut $(V_A, V - V_A)$, where $V_A =$ vertices that A is incident on. Add this edge to A .

Graph Algorithms: Prim's MST (pseudoC)

```
PRIM( $G, w, r$ )  
   $Q = \emptyset$   
  for each  $u \in G.V$   
     $u.key = \infty$   
     $u.\pi = \text{NIL}$   
    INSERT( $Q, u$ )  
  DECREASE-KEY( $Q, r, 0$ )      //  $r.key = 0$   
  while  $Q \neq \emptyset$   
     $u = \text{EXTRACT-MIN}(Q)$   
    for each  $v \in G.Adj[u]$   
      if  $v \in Q$  and  $w(u, v) < v.key$   
         $v.\pi = u$   
        DECREASE-KEY( $Q, v, w(u, v)$ )
```

Time Complexity (using binary Min-Heap):

$$O(|E|\log|V| + |V|\log|V|)$$

Single Source Shortest Path

INIT-SINGLE-SOURCE(V, s)

for each $v \in V$

do $d[v] \leftarrow \infty$

$\pi[v] \leftarrow \text{NIL}$

$d[s] \leftarrow 0$

RELAX(u, v, w)

if $d[v] > d[u] + w(u, v)$

then $d[v] \leftarrow d[u] + w(u, v)$

$\pi[v] \leftarrow u$

Single Source Shortest Path: Bellman-Ford

```
BELLMAN-FORD( $V, E, w, s$ )  
  INIT-SINGLE-SOURCE( $V, s$ )  
  for  $i \leftarrow 1$  to  $|V| - 1$   
    do for each edge  $(u, v) \in E$   
      do RELAX( $u, v, w$ )  
  for each edge  $(u, v) \in E$   
    do if  $d[v] > d[u] + w(u, v)$   
      then return FALSE  
  return TRUE
```

Core: The first **for** loop relaxes all edges $|V| - 1$ times

Time: $\Theta(VE)$.

Single Source Shortest Path: Dijkstra

Not applicable to graph with –ve weights

DIJKSTRA(V, E, w, s)

INIT-SINGLE-SOURCE(V, s)

$S \leftarrow \emptyset$

$Q \leftarrow V \quad \triangleright$ i.e., insert all vertices into Q

while $Q \neq \emptyset$

do $u \leftarrow \text{EXTRACT-MIN}(Q)$

$S \leftarrow S \cup \{u\}$

for each vertex $v \in \text{Adj}[u]$

do RELAX(u, v, w)

Dynamic Programming: Rod cutting

You are given a rod of length $n \geq 0$ (n in inches)

A rod of length i inches will be sold for p_i dollars

Cutting is free (simplifying assumption)

Problem: given a table of prices p_i determine the maximum revenue r_n obtainable by cutting up the rod and selling the pieces.

Length i	1	2	3	4	5	6	7	8	9	10
Price p_i	1	5	8	9	10	17	17	20	24	30

Dynamic Programming: Rod cutting

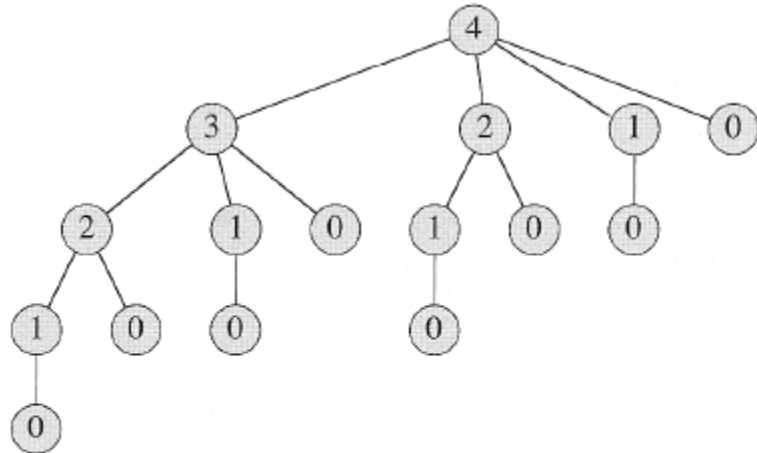
In ALL cases we have the recursion

$$r_n = \max(p_n, r_1 + r_{n-1}, r_2 + r_{n-2}, \dots, r_{n-1} + r_1)$$

■ Without Dynamic Programming

CUT-ROD(p, n)

```
1  if  $n == 0$ 
2      return 0
3   $q = -\infty$ 
4  for  $i = 1$  to  $n$ 
5       $q = \max(q, p[i] + \text{CUT-ROD}(p, n - i))$ 
6  return  $q$ 
```



The number of nodes for a tree corresponding to a rod of size n is:

$$T(0) = 1, \quad T(n) = 1 + \sum_{j=0}^{n-1} T(j) = 2^n, \quad n \geq 1.$$

Dynamic Programming; Rod Cutting

■ With Dynamic Programming

MEMOIZED-CUT-ROD(p, n)

```
1  let  $r[0..n]$  be a new array
2  for  $i = 0$  to  $n$ 
3       $r[i] = -\infty$ 
4  return MEMOIZED-CUT-ROD-AUX( $p, n, r$ )
```

MEMOIZED-CUT-ROD-AUX(p, n, r)

```
1  if  $r[n] \geq 0$ 
2      return  $r[n]$ 
3  if  $n == 0$ 
4       $q = 0$ 
5  else  $q = -\infty$ 
6      for  $i = 1$  to  $n$ 
7           $q = \max(q, p[i] + \text{MEMOIZED-CUT-ROD-AUX}(p, n - i, r))$ 
8   $r[n] = q$ 
9  return  $q$ 
```

Dynamic Programming: Matrix Multiplication

LOOKUP-CHAIN(m, p, i, j)

```

1  if  $m[i, j] < \infty$ 
2      return  $m[i, j]$ 
3  if  $i == j$ 
4       $m[i, j] = 0$ 
5  else for  $k = i$  to  $j - 1$ 
6       $q = \text{LOOKUP-CHAIN}(m, p, i, k)$ 
          +  $\text{LOOKUP-CHAIN}(m, p, k + 1, j) + p_{i-1}p_kp_j$ 
7      if  $q < m[i, j]$ 
8           $m[i, j] = q$ 
9  return  $m[i, j]$ 
    
```

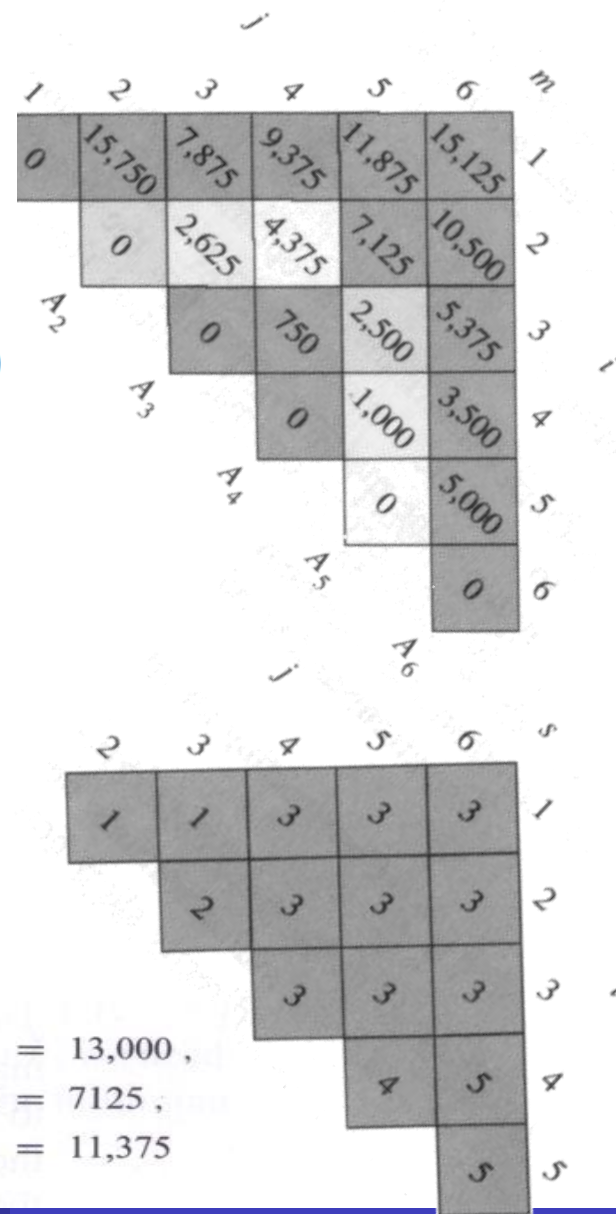
MATRIX-CHAIN(p)

```

1   $n = p.\text{length} - 1$ 
2  let  $m[1..n, 1..n]$  be a new table
3  for  $i = 1$  to  $n$ 
4      for  $j = i$  to  $n$ 
5           $m[i, j] = \infty$ 
6  return LOOKUP-CHAIN( $m, p, 1, n$ )
    
```

matrix	A_1	A_2	A_3	A_4	A_5	A_6
dimension	30×35	35×15	15×5	5×10	10×20	20×25

$$\begin{aligned}
 m[2, 5] &= \min \begin{cases} m[2, 2] + m[3, 5] + p_1 p_2 p_5 = 0 + 2500 + 35 \cdot 15 \cdot 20 = 13,000, \\ m[2, 3] + m[4, 5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \cdot 5 \cdot 20 = 7125, \\ m[2, 4] + m[5, 5] + p_1 p_4 p_5 = 4375 + 0 + 35 \cdot 10 \cdot 20 = 11,375 \end{cases} \\
 &= 7125.
 \end{aligned}$$



Matrix Multiplication: Bottom up, iterative

MATRIX-CHAIN-ORDER(p)

```
1   $n \leftarrow \text{length}[p] - 1$ 
2  for  $i \leftarrow 1$  to  $n$ 
3      do  $m[i, i] \leftarrow 0$ 
4  for  $l \leftarrow 2$  to  $n$        $\triangleright l$  is the chain length.
5      do for  $i \leftarrow 1$  to  $n - l + 1$ 
6          do  $j \leftarrow i + l - 1$ 
7               $m[i, j] \leftarrow \infty$ 
8              for  $k \leftarrow i$  to  $j - 1$ 
9                  do  $q \leftarrow m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j$ 
10                     if  $q < m[i, j]$ 
11                         then  $m[i, j] \leftarrow q$ 
12                              $s[i, j] \leftarrow k$ 
13 return  $m$  and  $s$ 
```

Dynamic Programming: LCS

LCS-LENGTH(X, Y)

```

1   $m = X.length$ 
2   $n = Y.length$ 
3  let  $b[1..m, 1..n]$  and  $c[0..m, 0..n]$  be new tables
4  for  $i = 1$  to  $m$ 
5       $c[i, 0] = 0$ 
6  for  $j = 0$  to  $n$ 
7       $c[0, j] = 0$ 
8  for  $i = 1$  to  $m$ 
9      for  $j = 1$  to  $n$ 
10         if  $x_i == y_j$ 
11              $c[i, j] = c[i - 1, j - 1] + 1$ 
12              $b[i, j] = \nwarrow$ 
13         elseif  $c[i - 1, j] \geq c[i, j - 1]$ 
14              $c[i, j] = c[i - 1, j]$ 
15              $b[i, j] = \uparrow$ 
16         else  $c[i, j] = c[i, j - 1]$ 
17              $b[i, j] = \leftarrow$ 
18  return  $c$  and  $b$ 
    
```

		j	0	1	2	3	4	5	6
		y_j		B	D	C	A	B	A
i	x_i								
0			0	0	0	0	0	0	0
1	A		0	$\uparrow$ 0	$\uparrow$ 0	$\uparrow$ 0	$\nwarrow$ 1	$\leftarrow$ 1	$\nwarrow$ 1
2	B		0	$\nwarrow$ 1	$\nwarrow$ 1	$\nwarrow$ 1	$\uparrow$ 1	$\nwarrow$ 2	$\leftarrow$ 2
3	C		0	$\uparrow$ 1	$\uparrow$ 1	$\nwarrow$ 2	$\nwarrow$ 2	$\uparrow$ 2	$\uparrow$ 2
4	B		0	$\nwarrow$ 1	$\uparrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\nwarrow$ 3	$\leftarrow$ 3
5	D		0	$\uparrow$ 1	$\nwarrow$ 2	$\uparrow$ 2	$\uparrow$ 2	$\nwarrow$ 3	$\uparrow$ 3
6	A		0	$\uparrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\nwarrow$ 3	$\uparrow$ 3	$\nwarrow$ 4
7	B		0	$\nwarrow$ 1	$\uparrow$ 2	$\uparrow$ 2	$\uparrow$ 3	$\nwarrow$ 4	$\nwarrow$ 4

PRINT-LCS(b, X, i, j)

```

1  if  $i == 0$  or  $j == 0$ 
2      return
3  if  $b[i, j] == \nwarrow$ 
4      PRINT-LCS( $b, X, i - 1, j - 1$ )
5      print  $x_i$ 
6  elseif  $b[i, j] == \uparrow$ 
7      PRINT-LCS( $b, X, i - 1, j$ )
8  else PRINT-LCS( $b, X, i, j - 1$ )
    
```

Approximation Algorithm: Subset Sum

Algorithm 1: EXACT-SUBSET-SUM(S, t)

```
1  $n \leftarrow |S|$ 
2  $L_0 \leftarrow \langle 0 \rangle$ 
3 for  $i = 1$  to  $n$  do
4    $L_i \leftarrow MergeLists(L_{i-1}, L_{i-1} + x_i)$ 
5   remove from  $L_i$  every element greater than  $t$ 
6 return the largest element in  $L_n$ 
```

Approximation Algorithm

The solution returned is within a factor of $1 + \epsilon$ of the optimal solution.

The running time is polynomial in both n and $1/\epsilon$

Algorithm 2: TRIM(L, δ)

```
1  $m \leftarrow |L|$ 
2  $L' \leftarrow \langle 0 \rangle$ 
3  $last \leftarrow y_1$ 
4 for  $i = 2$  to  $n$  do
5   if  $y_i > last \cdot (1 + \delta)$  then
6     append  $y_i$  onto the end of  $L'$ 
7      $last \leftarrow y_i$ 
8 return  $L'$ 
```

Algorithm 3: APPROX-SUBSET-SUM(S, t, ϵ)

```
1  $n \leftarrow |S|$ 
2  $L_0 \leftarrow \langle 0 \rangle$ 
3 for  $i = 1$  to  $n$  do
4    $L_i \leftarrow MergeLists(L_{i-1}, L_{i-1} + x_i)$ 
5    $L_i \leftarrow Trim(L_i, \epsilon/2n)$ 
6   remove from  $L_i$  every element greater than  $t$ ;
7 return the largest element in  $L_n$ 
```

Approximation Algorithm: Setcover

GREEDY-SET-COVER($X, \mathcal{F}$)

```
1   $U \leftarrow X$ 
2   $\mathcal{C} \leftarrow \emptyset$ 
3  while  $U \neq \emptyset$ 
4      do select an  $S \in \mathcal{F}$  that maximizes  $|S \cap U|$ 
5           $U \leftarrow U - S$ 
6           $\mathcal{C} \leftarrow \mathcal{C} \cup \{S\}$ 
7  return  $\mathcal{C}$ 
```

Note: TSP is also covered in the class, in addition we discussed, P,NP, NP complete and NP hard, cook Levin theorem, and idea of reduction